

Reviewer Pack — Minimal Number of Automorphic Forms and Resolution Proof Wid...

Entry 99116fd11123 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 12:43:32 UTC

Minimal Number of Automorphic Forms and Resolution Proof Width Correlation

Entry ID: 99116fd11123 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 12:43:32 UTC

1. Conjecture

Field A (mathematical branch): Automorphic Form Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given CNF φ with n variables, the minimal number of automorphic forms associated with the characteristic polynomials of the clauses in φ is linearly correlated with its resolution proof width $w(\varphi)$, such that the number of automorphic forms $N(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Automorphic forms encode symmetries in algebraic structures, which may reflect the underlying complexity of logical satisfiability. If this symmetry is reflected in the structure of CNFs, it could provide insights into the complexity of resolution proofs.

Taxonomy category: AUTOMORPHIC_FORMS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7d31ac23a1b6b8e8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal number of automorphic forms $N(\varphi)$ and the resolution proof width $w(\varphi)$ for each CNF φ is ≥ 0.8 , with a mean metric ≤ 3 across all seeds. The criterion is falsified if any seed produces a Pearson correlation coefficient < 0.5 or a mean metric > 10 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automorphic form AND resolution proof complexity - resolution proof width AND automorphic form count - CNF characteristic polynomial AND automorphic form theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def characteristic_polynomial(clause):
        n = len(clause)
        if n == 1:
            return [[-clause[0], 1]]
        matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for i, x in enumerate(clause):
            matrix[i][i] = -x
            matrix[n][i] = 1
            matrix[i][n] = 1
        det = determinant(matrix)
        return [[det, 1]]

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
        for j in range(n):
            submatrix = [[matrix[i][k] for k in range(n) if k != j] for i in range(1, n)]
            det += (-1) ** j * matrix[0][j] * determinant(submatrix)
        return det

    def resolution_width(phi):
        # Simplified DPLL solver to estimate resolution width
        clauses = phi[:]
        while True:
```

```

unit_clauses = [c for c in clauses if len(c) == 1]
if not unit_clauses:
    break
unit_clause = random.choice(unit_clauses)
literals = set(abs(lit) for lit in unit_clause)
new_clauses = []
for clause in clauses:
    if not any(lit in literals for lit in clause):
        continue
    if all(lit in literals for lit in clause):
        continue
    new_clause = [lit for lit in clause if abs(lit) not in literals]
    if new_clause:
        new_clauses.append(new_clause)
clauses = new_clauses
return len(clauses)

def automorphic_forms(char_polynomials):
    # Simplified counting of automorphic forms (placeholder)
    return sum(1 for cp in char_polynomials if cp[0] == 0)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    phi = [[random.randint(-n, n) for _ in range(n)] for _ in range(30)]
    char_polynomials = [characteristic_polynomial(clause) for clause in phi]
    w_phi = resolution_width(phi)
    N_phi = automorphic_forms(char_polynomials)
    results.append((N_phi, w_phi))

if not results:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

N_values, w_values = zip(*results)
mean_N = sum(N_values) / len(N_values)
mean_w = sum(w_values) / len(w_values)
correlation_coefficient = (sum((N - mean_N) * (w - mean_w) for N, w in results) /
    math.sqrt(sum((N - mean_N) ** 2 for N in N_values) *
        sum((w - mean_w) ** 2 for w in w_values)))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8 and mean_N <= 3,
    "counterexample": ""
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction=1")
    elif any(r['metric_value'] < 0.5 or r['metric_value'] > 10 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_out_of_bounds\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the Pearson correlation coefficient and mean metric could not be calculated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14336
2	preregistration	ollama_remote	glm4:latest	0	0	9615
3	novelty	ollama_remote	glm4:latest	0	0	8312
4	novelty	ollama_remote	glm4:latest	0	0	16484
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17003
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23323
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11954

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28833
9	judge	ollama_remote	glm4:latest	0	0	12149

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142011 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/99116fd11123.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/99116fd11123.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/99116fd11123.tar.gz (if generated)